Customer Churn Analysis: Why Customers Leave and What To Do About It

Business question: A telecom company is losing customers. Which customers are most likely to leave, why, and what should the business do to keep them?

Dataset: IBM Telco Customer Churn (7,043 customers, 21 columns), free on Kaggle. Download `WA_Fn-UseC_-Telco-Customer-Churn.csv` and put it in the same folder as this notebook.

Tools: Python, pandas, matplotlib, seaborn, scikit-learn

Flow: clean the data → find churn patterns → build a prediction model → turn the findings into business recommendations.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix, ConfusionMatrixDisplay

sns.set_theme(style="whitegrid")
pd.set_option("display.max_columns", None)

df = pd.read_csv("/cust1.csv")
print(df.shape)
df.head()
```

```
(7043, 21)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL	No
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL	Yes
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL	Yes
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL	Yes
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber optic	No

1. Clean the data

```
# TotalCharges is stored as text; blank values belong to brand-new customers (tenure = 0)
df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
print("Missing TotalCharges:", df["TotalCharges"].isna().sum())
df["TotalCharges"] = df["TotalCharges"].fillna(0)

# Turn the target into 1 (left) / 0 (stayed)
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})

# customerID is just a label, not useful for prediction
df = df.drop(columns=["customerID"])

print("Duplicate rows:", df.duplicated().sum())
df.info()
```

```
Missing TotalCharges: 11
Duplicate rows: 22
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 20 columns):
 #   Column            Non-Null Count  Dtype
```

```
---  ------            --------------  -----
 0   gender            7043 non-null   object
 1   SeniorCitizen     7043 non-null   int64
 2   Partner           7043 non-null   object
 3   Dependents        7043 non-null   object
 4   tenure            7043 non-null   int64
 5   PhoneService      7043 non-null   object
 6   MultipleLines     7043 non-null   object
 7   InternetService   7043 non-null   object
 8   OnlineSecurity    7043 non-null   object
 9   OnlineBackup      7043 non-null   object
 10  DeviceProtection  7043 non-null   object
 11  TechSupport       7043 non-null   object
 12  StreamingTV       7043 non-null   object
 13  StreamingMovies   7043 non-null   object
 14  Contract          7043 non-null   object
 15  PaperlessBilling  7043 non-null   object
 16  PaymentMethod     7043 non-null   object
 17  MonthlyCharges    7043 non-null   float64
 18  TotalCharges      7043 non-null   float64
 19  Churn             7043 non-null   int64
dtypes: float64(2), int64(3), object(15)
memory usage: 1.1+ MB
```

2. How big is the problem?

```
churn_rate = df["Churn"].mean()
print(f"Overall churn rate: {churn_rate:.1%}")
print(f"Monthly revenue from churned customers: {df.loc[df.Churn==1, 'MonthlyCharges'].sum():,.0f}")
print(f"Share of total monthly revenue lost: {df.loc[df.Churn==1, 'MonthlyCharges'].sum() / df['MonthlyCharges'].sum():.1%}")
```

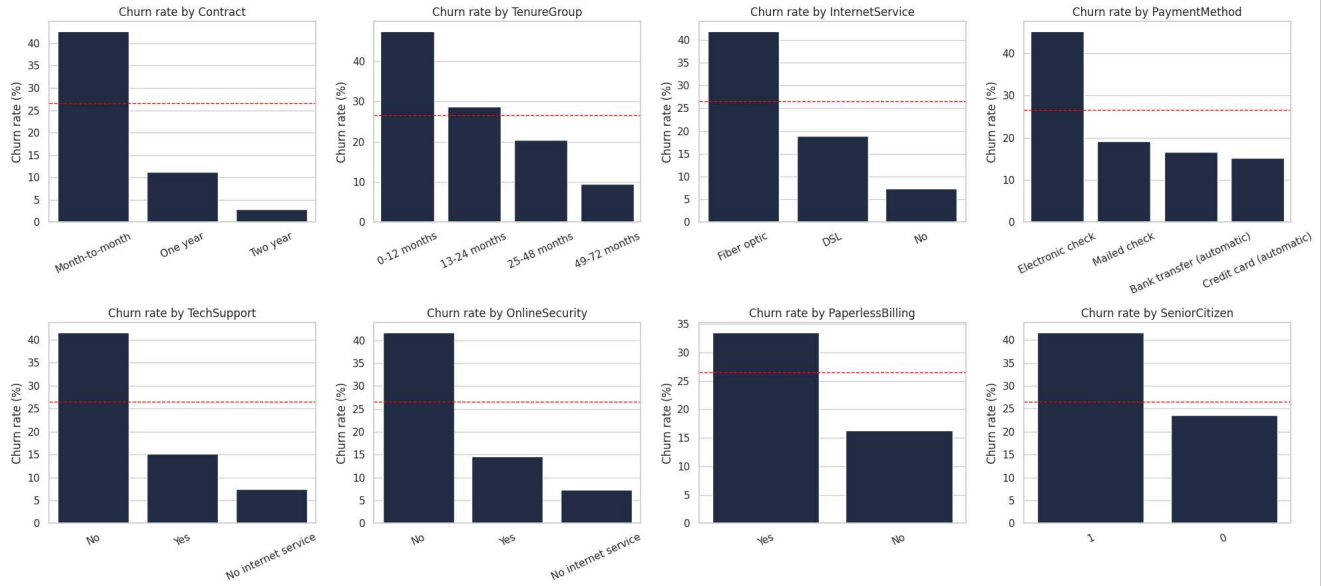
```
Overall churn rate: 26.5%
Monthly revenue from churned customers: 139,131
Share of total monthly revenue lost: 30.5%
```

3. Who is leaving? Churn by segment

```
# Group customers by how long they have stayed
df["TenureGroup"] = pd.cut(df["tenure"], bins=[-1, 12, 24, 48, 72],
                           labels=["0-12 months", "13-24 months", "25-48 months", "49-72 months"])

segments = ["Contract", "TenureGroup", "InternetService", "PaymentMethod",
            "TechSupport", "OnlineSecurity", "PaperlessBilling", "SeniorCitizen"]

fig, axes = plt.subplots(2, 4, figsize=(20, 9))
for ax, col in zip(axes.flat, segments):
    rate = df.groupby(col, observed=True)["Churn"].mean().sort_values(ascending=False) * 100
    sns.barplot(x=rate.index.astype(str), y=rate.values, ax=ax, color="#1B2A4A")
    ax.set_title(f"Churn rate by {col}")
    ax.set_ylabel("Churn rate (%)")
    ax.set_xlabel("")
    ax.tick_params(axis="x", rotation=25)
    ax.axhline(churn_rate * 100, color="red", linestyle="--", linewidth=1)
plt.tight_layout()
plt.savefig("churn_by_segment.png", dpi=150)
plt.show()
```




```
# Same story as numbers: the segments with the highest churn
for col in ["Contract", "TenureGroup", "InternetService", "PaymentMethod"]:
    print((df.groupby(col, observed=True)["Churn"].mean().sort_values(ascending=False) * 100).round(1).to_string())
    print()
```

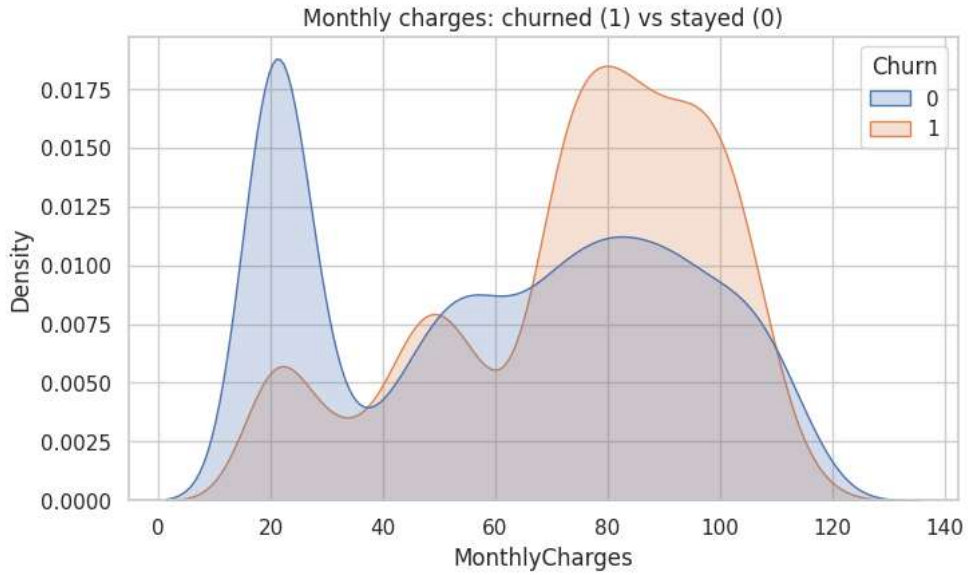
```
Contract
Month-to-month    42.7
One year          11.3
Two year           2.8

TenureGroup
0-12 months     47.4
13-24 months    28.7
25-48 months    20.4
49-72 months     9.5

InternetService
Fiber optic    41.9
DSL            19.0
No              7.4

PaymentMethod
Electronic check             45.3
Mailed check                 19.1
Bank transfer (automatic)    16.7
Credit card (automatic)      15.2
```

```
# Do churned customers pay more per month?
plt.figure(figsize=(8, 4.5))
sns.kdeplot(data=df, x="MonthlyCharges", hue="Churn", fill=True, common_norm=False)
plt.title("Monthly charges: churned (1) vs stayed (0)")
plt.savefig("monthly_charges_by_churn.png", dpi=150)
plt.show()
print(df.groupby("Churn")[["tenure", "MonthlyCharges", "TotalCharges"]].mean().round(1))
```




```
       tenure  MonthlyCharges  TotalCharges
Churn
0        37.6            61.3        2549.9
1        18.0            74.4        1531.8
```

4. Build a model to predict who will leave

```
X = df.drop(columns=["Churn", "TenureGroup"])
y = df["Churn"]

num_cols = X.select_dtypes(include="number").columns.tolist()
cat_cols = X.select_dtypes(exclude="number").columns.tolist()

preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced",
                                            min_samples_leaf=5, random_state=42),
}

results = {}
for name, model in models.items():
    pipe = Pipeline([("prep", preprocess), ("model", model)])
    pipe.fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    pred = (proba >= 0.5).astype(int)
    results[name] = {"pipe": pipe, "proba": proba, "pred": pred}
    print("=" * 60)
    print(name, "| ROC-AUC:", round(roc_auc_score(y_test, proba), 3))
    print(classification_report(y_test, pred, target_names=["Stayed", "Churned"]))
```

```
============================================================
Logistic Regression | ROC-AUC: 0.842
              precision    recall  f1-score   support

      Stayed       0.90      0.72      0.80      1035
     Churned       0.50      0.78      0.61       374

    accuracy                           0.74      1409
   macro avg       0.70      0.75      0.71      1409
weighted avg       0.80      0.74      0.75      1409

============================================================
Random Forest | ROC-AUC: 0.842
              precision    recall  f1-score   support
```

```
              precision    recall  f1-score   support

     Stayed       0.89      0.78      0.83      1035
    Churned       0.55      0.72      0.62       374

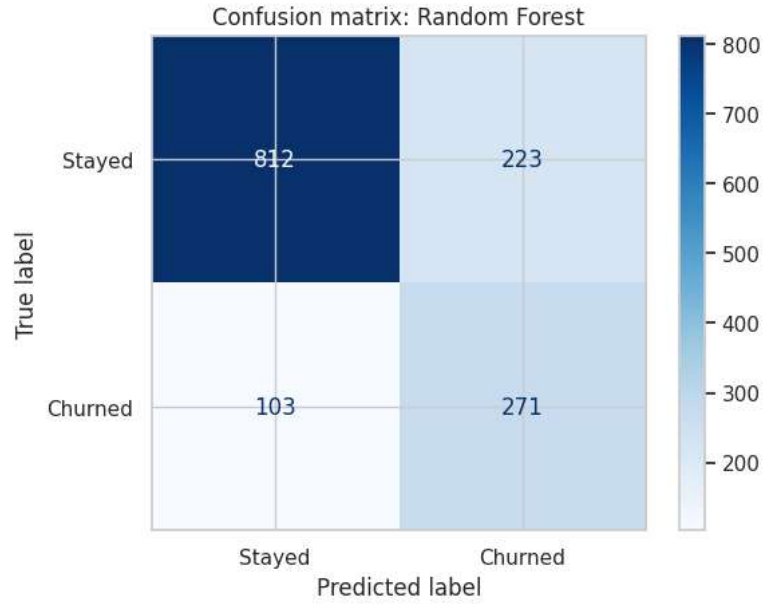
   accuracy                           0.77      1409
  macro avg       0.72      0.75      0.73      1409
weighted avg       0.80      0.77      0.78      1409
```

How to read this: for churn, *recall* on the "Churned" class matters most, because it shows how many of the customers who really left the model caught. A high-accuracy model that misses most churners is not useful.

```
best_name = max(results, key=lambda n: roc_auc_score(y_test, results[n]["proba"]))
print("Best model by ROC-AUC:", best_name)

ConfusionMatrixDisplay(confusion_matrix(y_test, results[best_name]["pred"]),
                       display_labels=["Stayed", "Churned"]).plot(cmap="Blues")
plt.title(f"Confusion matrix: {best_name}")
plt.savefig("confusion_matrix.png", dpi=150)
plt.show()
```

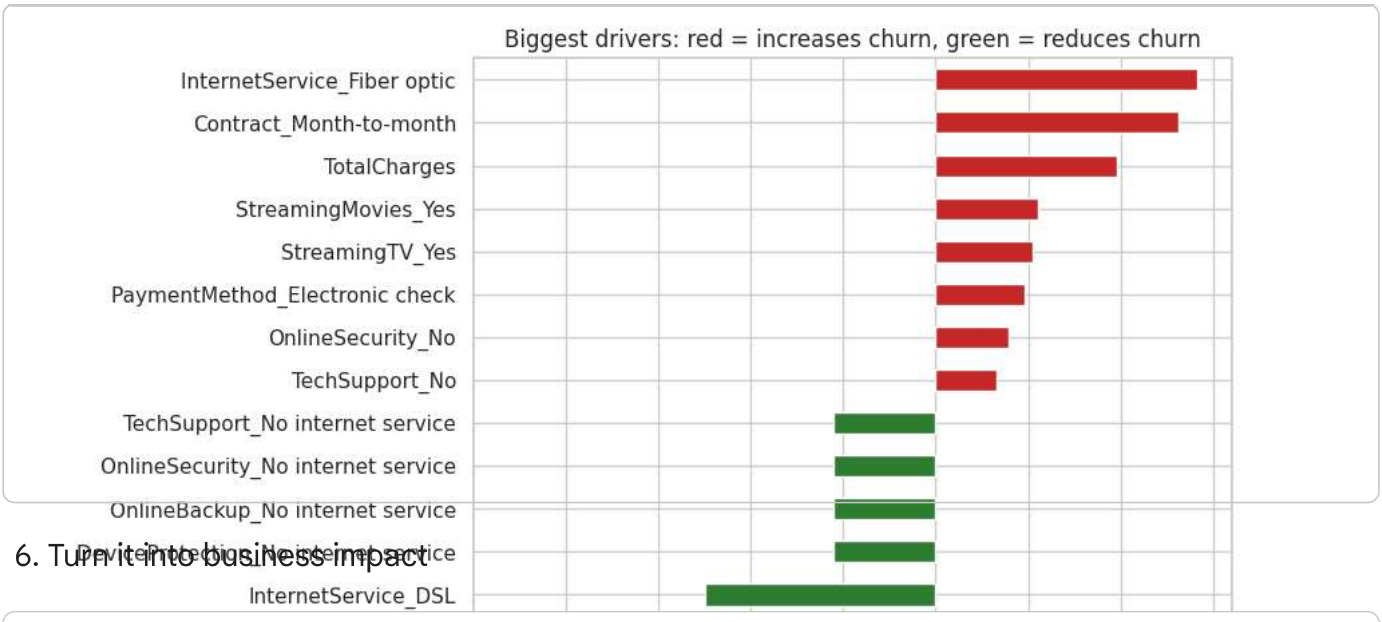
```
Best model by ROC-AUC: Random Forest
```



5. What drives churn? (the part the business cares about)

```
# Use the logistic regression coefficients: positive = pushes toward leaving, negative = keeps customers
lr_pipe = results["Logistic Regression"]["pipe"]
feat_names = lr_pipe.named_steps["prep"].get_feature_names_out()
coefs = pd.Series(lr_pipe.named_steps["model"].coef_[0], index=feat_names).sort_values()
coefs.index = [c.replace("num__", "").replace("cat__", "") for c in coefs.index]

top = pd.concat([coefs.head(8), coefs.tail(8)])
plt.figure(figsize=(9, 6))
top.plot(kind="barh", color=["#2E7D32" if v < 0 else "#C62828" for v in top.values])
plt.title("Biggest drivers: red = increases churn, green = reduces churn")
plt.tight_layout()
plt.savefig("churn_drivers.png", dpi=150)
plt.show()
```



6. Turn it into business impact

```
# Customers the model flags as high risk in the test set
test = X_test.copy()
test["Churn"] = y_test.values
test["risk"] = results[best_name]["proba"]
high_risk = test[test["risk"] >= 0.5]

at_risk_revenue = high_risk["MonthlyCharges"].sum()
print(f"High-risk customers flagged: {len(high_risk)}")
print(f"Monthly revenue at risk: {at_risk_revenue:,.0f}")

# ASSUMPTION (change this): a retention offer saves this share of flagged customers
save_rate = 0.20
print(f"If a retention offer saves {save_rate:.0%} of them: {at_risk_revenue * save_rate:,.0f} revenue kept per month")

# Who to contact first? Highest risk and highest bill
priority = high_risk.sort_values(["risk", "MonthlyCharges"], ascending=False)[
    ["Contract", "tenure", "InternetService", "PaymentMethod", "MonthlyCharges", "risk"]].head(10)
priority
```

```
High-risk customers flagged: 494
Monthly revenue at risk: 36,806
If a retention offer saves 20% of them: 7,361 revenue kept per month
```

	Contract	tenure	InternetService	PaymentMethod	MonthlyCharges	risk
6866	Month-to-month	1	Fiber optic	Electronic check	95.45	0.963511
4585	Month-to-month	1	Fiber optic	Electronic check	85.05	0.955823
6623	Month-to-month	1	Fiber optic	Electronic check	76.45	0.955227
809	Month-to-month	1	Fiber optic	Electronic check	69.55	0.954747
3346	Month-to-month	2	Fiber optic	Electronic check	84.05	0.953963
1731	Month-to-month	1	Fiber optic	Electronic check	69.60	0.952972
3682	Month-to-month	1	Fiber optic	Electronic check	69.10	0.950115
2927	Month-to-month	1	Fiber optic	Electronic check	69.90	0.949578
1739	Month-to-month	1	Fiber optic	Electronic check	69.90	0.949578
2797	Month-to-month	3	Fiber optic	Electronic check	100.95	0.949333

Next steps: Generate code with priority | New interactive sheet

7. Business recommendations

Fill this in using YOUR results from sections 3, 5 and 6. Keep each point to one line and always include a number.

1. **Biggest risk group:** [e.g. customers on ____ contracts with tenure under ____ months churn at ____%, versus ____% overall].
2. **Offer to test:** [e.g. move month-to-month customers to a 1-year plan with a discount].
3. **Service fix:** [e.g. customers without ____ churn ____% more, so bundle it for the first ____ months].